MINUTES OF MEETING (MOM)
Team 8
3D Printing Cost Estimation Platform

Meeting Details

Project:	3D Printing Cost Estimation Platform
Week:	Week 2
Date:	Thursday, January 23, 2026
Time:	4:00 PM IST
Location:	CIE, Ground Floor (Internal Team Meeting)
Client:	Anil Kumar Upadhyay, Five Fingers Innovative Solutions
Client Email:	aku1974india@gmail.com
Client Phone:	9949497560

Team Members Present

- Abhiroop Pullepu
- Mehrish Khan
- Ronith Menneni
- Sai Hasini
- Ruschita Guddeti

Notes

- **Client was unavailable for Week 2 meeting due to scheduling conflicts.**
- Team conducted internal planning meeting to discuss project structure and technology stack.
- Focus on repository setup, documentation standards, and task distribution.

1 GitHub Repository Setup

1.1 Repository Structure

- Repository name: `3d-printing-cost-estimator`
- Organization: Team 8 GitHub organization
- Main branch: `main` (protected, requires PR approval)
- Development branch: `dev` (baseline code for all feature development)
- Branch naming convention: `feature/<feature-name>`, `bugfix/<issue-id>`

1.2 Branching Strategy

- **main** branch: Production-ready code with release tags
- **dev** branch: Integration branch for feature development
- **Feature branches:** Created from `dev`, merged back via Pull Requests
- **Pull Request workflow:**
  1. Create feature branch from `dev`
  2. Implement and test locally
  3. Create PR with description and linked Jira issue
  4. Team lead reviews and approves
  5. Merge to `dev` after CI passes

1.3 Initial Repository Contents

- README.md with project overview
- .gitignore for Python and Node.js
- LICENSE file (MIT)
- Project directory structure skeleton
- Docker Compose configuration template

2 Jira Workspace Configuration

2.1 Project Setup

- Project name: **3D Printing Platform - Team 8**
- Project key: `3DP`
- Project type: Scrum
- Sprint duration: 1 week (Thursday to Wednesday)

2.2 Epic Structure

1. **3DP-EPIC-1:** Backend API & CURA Integration
2. **3DP-EPIC-2:** Frontend Development & 3D Visualization
3. **3DP-EPIC-3:** Database Schema & ORM
4. **3DP-EPIC-4:** DevOps & CI/CD Pipeline
5. **3DP-EPIC-5:** Testing & Documentation

2.3 Issue Types & Workflow

- **Story:** User-facing features (e.g., "As a user, I can upload STL files")
- **Task:** Technical implementation (e.g., "Setup PostgreSQL database")
- **Bug:** Defects found during testing
- **Sub-task:** Breakdown of stories/tasks
- **Workflow:** To Do → In Progress → Code Review → Testing → Done

2.4 GitHub-Jira Integration

- Install Jira GitHub integration app
- Link commits to issues using format: `3DP-<issue-number>: <commit message>`
- Automatic status updates on PR creation/merge
- View commits and PRs directly in Jira issues

3 Documentation Standards

3.1 Architecture Diagrams

- **Use Case Diagram:** Actors and system interactions
- **Control Flow Diagram:** High-level system flow (upload → slice → estimate)
- **Sequence Diagram:** Detailed interactions for key use cases
- **Tool:** Mermaid.js (stored in `/docs/diagrams/` as Markdown)

3.2 API Documentation

- Swagger/OpenAPI 3.0 specification
- Auto-generated from FastAPI route decorators
- Accessible at `/docs` endpoint
- Include request/response examples for all endpoints

3.3 Code Documentation

- Python: Docstrings for all functions/classes (Google style)
- TypeScript: TSDoc comments for public interfaces
- README files in each major directory
- Inline comments for complex logic only

4 Technology Stack Finalization

4.1 Backend

- **Language:** Python 3.11+
- **Framework:** FastAPI 0.104+
- **Database:** PostgreSQL 15+ with SQLAlchemy ORM
- **Testing:** pytest, pytest-asyncio
- **Linting:** black (formatter), pylint (linter)

4.2 Frontend

- **Framework:** React 18+ with TypeScript
- **3D Library:** Three.js r150+
- **UI Library:** Material-UI 5+
- **Build Tool:** Vite 4+
- **Testing:** React Testing Library, Vitest

4.3 DevOps

- **Containerization:** Docker & Docker Compose
- **CI/CD:** GitHub Actions
- **Hosting:** Local deployment (client will handle production)

5 CI/CD Pipeline Design

5.1 GitHub Actions Workflows

5.1.1 Backend CI (`.github/workflows/backend-ci.yml`)

- Trigger: Push/PR to `dev` or `main` branches
- Steps:
  1. Checkout code
  2. Setup Python 3.11
  3. Install dependencies from `requirements.txt`
  4. Run `black --check` (format check)
  5. Run `pylint` (linting)
  6. Run `pytest` with coverage report
  7. Upload coverage to Codecov (optional)

5.1.2 Frontend CI (`.github/workflows/frontend-ci.yml`)

- Trigger: Push/PR to `dev` or `main` branches
- Steps:
  1. Checkout code
  2. Setup Node.js 18+
  3. Install dependencies (`npm ci`)
  4. Run `npm run lint`
  5. Run `npm run type-check` (TypeScript)
  6. Run `npm run test` (unit tests)
  7. Build production bundle (`npm run build`)

5.1.3 Docker Build Check

- Verify `docker-compose.yml` builds successfully
- Run integration tests against Docker containers
- Ensure all services start without errors

5.2 Branch Protection Rules

- **main branch:**
  - Require PR before merge
  - Require 1 approval from team lead
  - Require CI checks to pass
  - No direct pushes allowed
- **dev branch:**
  - Require PR before merge
  - Require CI checks to pass
  - Allow team members to merge after review

6 Task Distribution

6.1 Individual Responsibilities

- **Abhiroop Pullepu (Backend Lead):**
  - CURA slicing engine integration
  - G-code parsing and analysis
  - Failure logging system
  - Swagger API documentation
- **Mehrish Khan (Backend Developer):**
  - File upload API endpoints
  - Cost calculation engine
  - Database operations and migrations
  - Email notification service (Mailtrap)
- **Ruschita Guddeti (Project Manager / Integration Lead):**

  - Database schema design
  - GitHub-Jira integration setup
  - Architecture and sequence diagrams
  - Sprint coordination and burndown tracking
  - Release management

- **Sai Hasini (Frontend Developer):**
  - File upload UI with drag-and-drop
  - CURA slicing parameters form
  - Form validation and error handling
  - User notifications and feedback

- **Ronith Menneni (Frontend Developer):**
  - Three.js 3D viewer implementation
  - STL model rendering with OrbitControls
  - G-code layer visualization
  - Interactive model manipulation (rotate/zoom/pan)

6.2 Contribution Tracking

- All commits must reference Jira issue IDs
- Use GitHub Insights to track contribution metrics
- Weekly code review sessions to ensure quality
- Each member must have at least 2 merged PRs per sprint

7 Release Strategy

7.1 Versioning Scheme

- Format: `vX.Y.Z` (Semantic Versioning)
- **X (Major):** Breaking API changes
- **Y (Minor):** New features, backward compatible
- **Z (Patch):** Bug fixes, hotfixes

7.2 Release Workflow

1. Complete all stories in sprint
2. Merge feature branches to `dev`
3. Run full test suite on `dev` branch
4. Create release branch: `release/vX.Y.Z`
5. Perform QA testing on release branch
6. Fix critical bugs in release branch
7. Merge release branch to `main`

8. Tag commit in `main`: `git tag -a vX.Y.Z -m "Release notes"`
9. Push tag: `git push origin vX.Y.Z`
10. Merge `main` back to `dev`
11. Generate release notes

7.3 Release Notes Template

```
Release vX.Y.Z - Date

New Features:
- [3DP-XX] Feature description
- [3DP-YY] Feature description

Bug Fixes:
- [3DP-ZZ] Bug fix description

Known Issues:
- Issue description (if any)

Testing:
- X unit tests passed
- Y integration tests passed
- Manual QA: PASS/FAIL
```

7.4 Hotfix Process

- Create hotfix branch from `main`: `hotfix/vX.Y.Z+1`
- Fix critical bug
- Test thoroughly
- Merge to `main` and tag
- Merge back to `dev`
- Update release notes

8 Testing Strategy

8.1 Test Levels

- **Unit Tests:** Individual functions/components (¿80% coverage)
- **Integration Tests:** API endpoints with database
- **E2E Tests:** Complete workflows (upload → slice → estimate)
- **Manual QA:** UI/UX validation before release

8.2 Test Documentation

- Test cases documented in Jira as "Test" issue type
- Link test cases to user stories
- Track test results in Jira "Test Execution" field
- Failed tests create linked bug issues

9 Issue Management Process

9.1 Creating Issues

- All work must have corresponding Jira issue
- Include clear description and acceptance criteria
- Assign story points (Fibonacci: 1, 2, 3, 5, 8)
- Link to relevant epic
- Add labels (backend, frontend, database, urgent, etc.)

9.2 Issue Lifecycle

1. **Created:** Issue added to backlog
2. **Sprint Planning:** Issue assigned to sprint
3. **In Progress:** Developer starts work, creates branch
4. **Code Review:** PR created, awaiting review
5. **Testing:** Merged to `dev`, QA testing
6. **Done:** Tests pass, issue closed

9.3 Blocking Issues

- Mark as "Blocker" priority
- Document blocker in Jira comments
- Escalate to team lead immediately
- Update manager in weekly status report

10 Communication Protocols

10.1 Daily Standups

- Time: 10:00 AM IST (15 minutes max)
- Format: What I did yesterday, what I'll do today, any blockers
- Update Jira status before standup

10.2 Weekly Client Meetings

- Time: Thursdays, 4:00 PM IST
- Prepare demo of completed features
- Document meeting in MOM
- Raise clarifications and get approvals

10.3 Sprint Retrospectives

- Held after sprint ends (Wednesdays)
- Discuss: What went well, what didn't, action items
- Document in Jira Sprint Report

11 Next Steps

11.1 Week 3 Goals

- Complete GitHub repository baseline setup
- Create all Jira epics and initial stories
- Implement GitHub Actions CI pipeline
- Setup Docker Compose for local development
- Create architecture and sequence diagrams
- Draft questions for client regarding CURA API

11.2 Pending from Week 1

- Awaiting CURA API specifications from client (Due: Jan 20)
- Awaiting material cost data from client (Due: Jan 20)
- Awaiting printer volume specifications (Due: Jan 20)

Minutes prepared by: Team 8
Date: January 23, 2026